BEE 4750 Lab 2: Linear Programming with JuMP

2026-10-21

> **Due Date**
>
> Tuesday, October 27, 9:00pm

Setup

Run these two cells first. The first time, installing and loading `JuMP` and `HiGHS` can take a few minutes.

```
import Pkg
Pkg.activate(".")
Pkg.instantiate()
```

```
using JuMP    # optimization modeling syntax
using HiGHS   # the solver
```

Introduction

In the October 19 lecture, we saw how to formulate and solve a two-stack emissions problem in `JuMP` and read its shadow prices. In this lab you will do the same for three power plants on a hot summer afternoon: write its linear program in `JuMP`, solve it, read and interpret the shadow prices, and re-solve it when one limit tightens.

The Plants

A utility owns three power plants side by side on a river: a coal steam plant, a combined-cycle plant and a combustion-turbine plant. It sells all the electricity they generate at the wholesale price, $120/MWh this afternoon, and decides how much each plant should generate this hour to make the most profit. Each plant's profit per MWh is the price minus its variable cost (mostly fuel).

Plant	Capacity (MW)	Variable cost ($/MWh)	Heat to river (MWth per MW)	NOx (lb/MWh)
Coal steam	600	30	1.5	1
Combined cycle	400	56	0.64	0.1
Combustion turbines	300	87	0	0.5

The three plants share three limits:

- **River heat.** The coal plant and the combined-cycle plant are cooled with river water, which goes back into the river warmer. On a hot day the river is already warm, so the state caps the heat the three plants together may discharge to this stretch of the river: 1,000 MWth this hour. (MWth is megawatts of heat, to keep it apart from the MW of electricity the plants generate.) The combustion turbines are air-cooled and send no heat to the river.
- **NOx.** Hot, sunny days are also ozone days, and NOx is one of the pollutants that forms ozone. Together, the three plants may emit at most 650 lb of NOx this hour.
- **Transmission line.** The plants share one line to the grid, which can carry at most 1,000 MW.

The same data in code:

```
plant_names = ["Coal steam", "Combined cycle", "Combustion turbines"]
n_plants = length(plant_names)

price = 120                      # electricity price          [$/MWh]
variable_cost = [30, 56, 87]     # fuel and operating cost    [$/MWh]
capacity = [600, 400, 300]       # maximum output             [MW]
heat_to_river = [1.5, 0.64, 0]   # heat sent to the river     [MWth per MW]
nox_rate = [1, 0.1, 0.5]         # NOx emitted                [lb/MWh]

heat_limit = 1000                # river heat permit          [MWth]
nox_limit = 650                  # NOx limit                  [lb]
```

```
line_limit = 1000                # transmission line capacity  [MW]
```

Two Ways to Work Through This Lab

You can do the coding either in this notebook (`lab02.ipynb`) or in the plain script `lab02.jl`, whichever you prefer: both contain the same problems. Either way, the written parts go on the separate **Activity Sheet** (handed out in class). Submit one PDF with your code output and your completed Activity Sheet to Gradescope by 9:00pm on Tuesday, October 27.

JuMP Syntax

From the October 19 lecture, a `JuMP` model has three kinds of pieces after the model itself:

```
model = Model(HiGHS.Optimizer)
@variable(model, 0 <= amount[i in 1:n_items] <= upper_bound[i])
@objective(model, Max, sum(unit_value[i] * amount[i] for i in 1:n_items))
@constraint(model, budget,
    sum(unit_cost[i] * amount[i] for i in 1:n_items) <= available)
```

Use `Max` or `Min` as the problem needs. The name in `@constraint` (here `budget`) is how you ask about that constraint later. After `optimize!(model)`:

- `value(amount[1])`, or `value.(amount)` for all of them, gives the optimal decisions;
- `objective_value(model)` gives the optimal objective;
- `value(budget)` gives the constraint's left-hand side at the optimum;
- `shadow_price(budget)` gives the change in the objective when `budget` is relaxed by one unit.

Put the whole model in one cell and re-run that whole cell after any change. `JuMP` will not add a second constraint with the same name to a model, so re-running a single `@constraint` line gives an error.

Problem 1: Formulate the LP

Before writing any code, complete Problem 1 on the Activity Sheet: write the utility's objective and the three shared limits as a linear program, with units. Use y_1, y_2 and y_3 for the output of the coal plant, the combined-cycle plant and the combustion-turbine plant this hour, in MW. Each is at least 0 and at most its plant's capacity.

Problem 2: Solve It in JuMP

Complete the model below from your Problem 1 formulation. The capacities are already in the `@variable` bounds. Write each of the three shared limits as a **named** constraint, using the names in the comments: the names are what let `JuMP` report their shadow prices in Problem 3.

```
utility = Model(HiGHS.Optimizer)
set_silent(utility)

# Output of each plant this hour [MW], between 0 and the plant's capacity
@variable(utility, 0 <= generation[i in 1:n_plants] <= capacity[i])

# TODO: the objective, the utility's profit this hour [$]
# @objective(utility, Max, ...)

# TODO: the three shared limits, each as a named constraint
# @constraint(utility, river_heat, ...)
# @constraint(utility, nox_emissions, ...)
# @constraint(utility, transmission_line, ...)

print(utility)
```

Check the printed model against your Activity Sheet, then solve it. The status should be `OPTIMAL`.

```
optimize!(utility)
termination_status(utility)
```

```
for i in 1:n_plants
    println(plant_names[i], ": ", value(generation[i]), " MW")
end
println("Profit: \$", objective_value(utility))
```

If your model matches the problem, the profit is $73,672 for the hour. If you get a different number, compare `print(utility)` with your Problem 1 formulation, or ask the TA.

How much of each limit does this plan use?

```
println("River heat: ", value(river_heat), " of ", heat_limit, " MWth")
println("NOx: ", value(nox_emissions), " of ", nox_limit, " lb")
println("Line: ", value(transmission_line), " of ", line_limit, " MW")
```

Record the plan in Problem 2 on the Activity Sheet and answer the question there.

Problem 3: Shadow Prices

Use `shadow_price` to get the shadow price of each of the three named constraints.

```
# TODO: the shadow price of each named constraint, for example
# println("River heat: ", shadow_price(river_heat))
```

`JuMP`'s `shadow_price` is the change in the objective when a constraint is relaxed by one unit. This model maximizes profit, and relaxing a limit can only raise the best profit or leave it unchanged, so expect shadow prices that are positive or zero. (In the cost-minimizing emissions problem from lecture they were negative or zero, for the same reason.)

Complete Problem 3 on the Activity Sheet.

Problem 4: A Hotter River

The heat wave deepens: the river warms further, and the state cuts the plants' heat limit from 1,000 to 850 MWth. Answer Problem 4a on the Activity Sheet, a prediction from the shadow price, **before** you re-solve.

First, save the results under the current limit, since re-solving will replace them:

```
base_generation = value.(generation)
base_profit = objective_value(utility)
base_nox = value(nox_emissions)
```

Now lower the heat limit and solve again. `set_normalized_rhs` changes a constraint's right-hand side in the model you already built, as in the October 19 lecture.

```
heatwave_heat_limit = 850   # [MWth]

# TODO: lower the river heat limit to heatwave_heat_limit, then solve again
# set_normalized_rhs(river_heat, ...)
# optimize!(utility)
```

```
for i in 1:n_plants
    println(plant_names[i], ": ", base_generation[i], " MW before, ",
        value(generation[i]), " MW after")
end
println("Profit: \$", base_profit, " before, \$",
    objective_value(utility), " after")
println("NOx: ", base_nox, " lb before, ", value(nox_emissions), " lb after")
```

Complete Problem 4 on the Activity Sheet.

If You Finish Early (Optional)

How far does the heat limit's shadow price hold? Re-solve at each heat limit below and store its shadow price. Where does it change, and why?

```
heat_limits = [1100, 1000, 900, 800, 700, 600]   # [MWth]
heat_prices = zeros(length(heat_limits))
for k in 1:length(heat_limits)
    # TODO: set the heat limit to heat_limits[k], solve again,
    # and store shadow_price(river_heat) in heat_prices[k]
end
heat_prices
```

In this lab the price of \$120/MWh was given. On Wednesday, the economic dispatch lecture shows where a price like that comes from.